Reviewer Pack — Minimal Rank of Formal Power Series over p-Adic Fields Bound…

Entry 2554166da828 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 01:37:24 UTC

Minimal Rank of Formal Power Series over p-Adic Fields Bounds Resolution Proof Length

Entry ID: `2554166da828` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 01:37:24 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory (Formal Power Series) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For every CNF instance F with m clauses and n variables, the minimal rank of a formal power series over the p-adic numbers associated with the clause indicators is upper bounded by the resolution proof length of F.', 'Equivalently, if R(p, F) denotes the formal power series over the p-adic field with minimal rank that can represent the clause indicators of F, then R(p, F) = O(log t*(F)), where t*(F) is the minimum depth of a resolution tree for F.']

Rationale (proposer's reasoning):

> ['Formal power series over p-adic fields provide a rich algebraic structure to study complexity invariants. The conjecture suggests that this structure can be leveraged to bound the length of resolution proofs, which are central to the complexity of satisfiability problems.', 'The use of p-adic formal power series may reveal new insights into the complexity of reasoning about logical formulas, potentially leading to better proof systems or lower bounds.']

Taxonomy category: `resolution_proof_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cc298dc7dfdc259f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean correlation coefficient between the minimal rank of the formal power series and the resolution proof length for all seeds exceeds 0.5, with p-value < 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic number theory" AND "formal power series" AND "resolution proof complexity" - "minimal rank" formal power series p-adic fields resolution proof length" - "formal power series over p-adic numbers" AND resolution proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def resolution_length(cnf):
        # Simplified DPLL solver to estimate resolution length
        stack = []
        while stack:
            literal = stack.pop()
            if literal in cnf:
                cnf.remove(literal)
            elif -literal in cnf:
                cnf.remove(-literal)
            else:
                return len(stack) + 1
        return len(stack)

    def formal_power_series_rank(cnf):
        # Placeholder for actual computation of rank
        return random.randint(1, 10)  # Simplified for testing
```

```
    n = random.randint(5, 40)
    m = random.randint(n, n * 2)
    cnf = generate_cnf(n, m)

    rank = formal_power_series_rank(cnf)
    length = resolution_length(cnf)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": rank / (length + 1),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(res["metric_value"] for res in results)
    mean_metric_value = total_metric_value / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fractio
    elif any(not res["conjecture_holds"] for res in results):
        counterexample = next(res for res in results if not res["conjecture_holds"])["counterexamp
        first_failing_seed = next(res for res in results if not res["conjecture_holds"])["seed"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
olds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 5.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 1.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 6.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 4.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 1.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 10.0, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 9.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 1.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 3.0, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 5.0, 'instances_tested': 1, 'conje
```

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_17a969fc.py", line 79, in <module>
    first_failing_seed = next(res for res in results if not res["conjecture_holds"])["seed"]
                         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which prevents us from evaluating the mean correlation coefficient and p-value. | next: Investigate the cause of the crash and rerun the test to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12476
2	propose	ollama_remote	glm4:latest	0	0	10671
3	preregistration	ollama_remote	glm4:latest	0	0	5446
4	novelty	ollama_remote	glm4:latest	0	0	4765
5	novelty	ollama_remote	glm4:latest	0	0	5572
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12556
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9135
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9972
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27493
10	judge	ollama_remote	glm4:latest	0	0	63901

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 161988 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2554166da828.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2554166da828.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2554166da828.tar.gz` (if generated)